

FINAL-TERM STUDY HANDOUT

Kubernetes • Deployments & Services • Persistent Storage • DevOps vs MLOps
DevOps and CC Foundations (CSC-341) Spring-2026

How to use this handout

This handout teaches every concept the final exam will test. It is built around a running example a video-streaming platform called “FlixHub” which is deliberately different from the exam’s case study so you learn the principles rather than memorizing answers.

If you can explain each concept here in your own words and reproduce the example manifests, you will be able to attempt every question on the paper. Worked examples here are templates: in the exam you must adapt the same reasoning to a new scenario.

End-of-section “**Check yourself**” prompts mirror the style of exam questions practice answering them from memory.

Contents

1. Kubernetes & Its Architecture.....	3
1.1 What Kubernetes is and why it exists	3
1.2 The cluster: control plane vs worker nodes.....	3
1.3 Walkthrough: what happens when you apply a manifest.....	4
1.4 Self-healing: how a node failure is handled.....	4
2. Deployments, ReplicaSets & Services	6
2.1 The object hierarchy: Pod → ReplicaSet → Deployment.....	6
2.2 The selector the link a Deployment must have.....	6
2.3 Controlling rolling updates: strategy block.....	6
2.4 Services giving Pods a stable address	7
3. Persistent Storage: PV, PVC & StorageClass	9
3.1 Why Pods need external storage	9
3.2 The three objects and who owns them	9
3.3 Worked example: attaching a disk to a database Pod.....	9
3.4 Access modes who can mount, and how	10
3.5 Reclaim policies what happens to data when a PVC is deleted	10
4. DevOps vs MLOps	12
4.1 Definitions.....	12
4.2 How MLOps differs from traditional DevOps	12
4.3 Why a model degrades without any code change: drift.....	12
4.4 What an MLOps pipeline adds beyond standard CI/CD	12

5. One-Page Quick Revision	14
----------------------------------	----

1. Kubernetes & Its Architecture

1.1 What Kubernetes is and why it exists

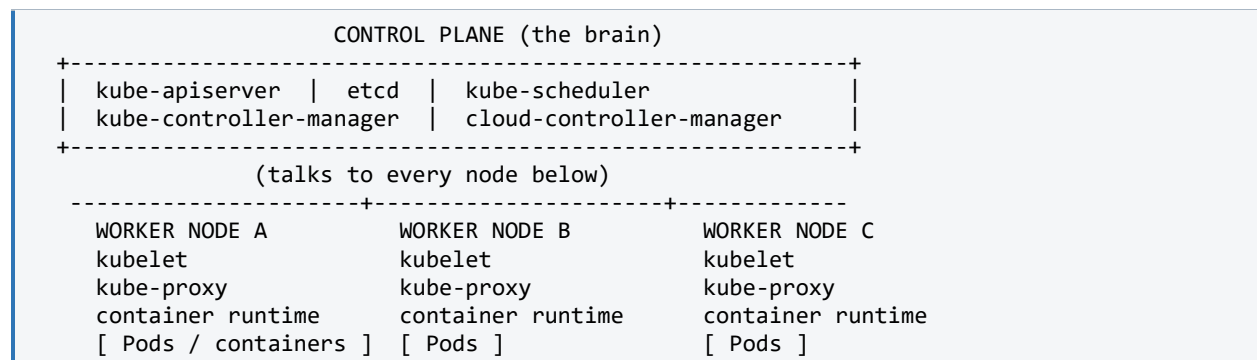
Containers package an application with its dependencies so it runs the same everywhere. But running many containers across many machines by hand is impractical: you need to decide which machine runs what, restart failed containers, scale up under load, and roll out new versions safely. Kubernetes (K8s) is a container orchestrator that automates all of this. You declare the desired state (“I want 3 copies of this app running”) and Kubernetes continuously works to make reality match that declaration.

The single most important idea: declarative + control loops

You never tell Kubernetes the steps to take. You declare what you want in a YAML manifest and apply it. Controllers run endless loops comparing desired state (what you declared) with actual state (what is running) and take corrective action. This is why a deleted pod comes back automatically a controller notices the gap and fixes it.

1.2 The cluster: control plane vs worker nodes

A Kubernetes cluster is split into two kinds of machines. The control plane is the brain that makes global decisions; the worker nodes are the muscle that actually run your application containers. Understanding which component lives where, and what each does, is essential.



Control-plane components (memorise the role of each)

Component	Responsibility
kube-apiserver	The front door and central hub. Every command (<code>kubectl</code> , controllers, kubelets) goes through it. It validates requests and is the ONLY component that reads/writes <code>etcd</code> .
etcd	A distributed key-value database holding the entire cluster state every object, spec, and secret. It is the cluster's single source of truth. Lose <code>etcd</code> (with no backup) and the cluster forgets everything it was supposed to run.
kube-scheduler	Watches for Pods that have no node yet and decides which node each should run on, based on available resources, affinity rules, taints/tolerations, etc.

Component	Responsibility
kube-controller-manager	Runs the control loops: node controller, replication/replicaset controller, endpoints controller, and more: each driving actual state toward desired state.
cloud-controller-manager	Talks to the cloud provider: provisions external load balancers, attaches storage volumes, and manages the cloud side of node lifecycle.

Worker-node components

Component	Responsibility
kubelet	The node agent. Watches for Pods assigned to its node and tells the container runtime to start them. Sends periodic heartbeats (node status) up to the apiserver.
kube-proxy	Programs the node's network rules so Service traffic is routed to the correct backing Pods.
container runtime	The software (e.g. containerd, CRI-O) that actually pulls images and runs containers.

etcd: why it is special (high-value exam point)

etcd stores the desired state of every object in the cluster. If it is lost and there is no backup, no component knows what should be running: the cluster cannot be reconstructed. This is why etcd is always backed up and often run as an odd-numbered HA cluster (3 or 5 members) for quorum. Whenever a question asks 'what is catastrophic about losing X', etcd is the answer.

1.3 Walkthrough: what happens when you apply a manifest

Tracing the lifecycle of `kubectl apply -f app.yaml` ties all the components together. Learn this ordered chain: you should be able to name which component does each step:

1. `kubectl` sends the manifest to the `kube-apiserver`.
2. The `apiserver` authenticates and validates it, then persists the object to `etcd`.
3. The `controller-manager` (deployment + replicaset controllers) sees the new object and creates the `ReplicaSet` and `Pod` objects: still unscheduled.
4. The `scheduler` sees `Pods` with no node, picks suitable nodes, and writes the binding back via the `apiserver`.
5. The `kubelet` on each chosen node tells the container runtime to pull the image and start the containers.
6. The `kube-proxy` updates network rules so the `Pods` are reachable through their `Service`.

1.4 Self-healing: how a node failure is handled

Because the `kubelet` sends regular heartbeats, the control plane knows when a node stops responding. The sequence is:

- The `kubelet`'s heartbeats to the `apiserver` stop arriving.

- After a grace period, the node controller marks the node NotReady and its Pods for eviction.
- The replicaset controller sees fewer running replicas than desired and creates replacement Pods.
- The scheduler places those replacements on the remaining healthy nodes desired state restored, automatically.

Check yourself Section 1

Draw the cluster split into control plane and worker nodes and place every component on the correct side from memory.

For each control-plane component, state its one-line job. Explain precisely why losing etcd is catastrophic.

Narrate the full apply→running-pod chain, naming the component at each handoff.

Explain, component by component, how the cluster recovers when one worker node dies.

2. Deployments, ReplicaSets & Services

2.1 The object hierarchy: Pod → ReplicaSet → Deployment

- **Pod** – the smallest deployable unit; one or more containers sharing network and storage. Pods are disposable you rarely create them directly.
- **ReplicaSet** – keeps a specified number of identical Pod replicas running. If one dies, it makes another; if there are too many, it deletes some.
- **Deployment** – a higher-level controller that manages ReplicaSets for you and adds rolling updates and rollbacks. This is what you normally write.

What happens on an image update (rolling update mechanics)

When you change a Deployment's image (say v1→v2), the Deployment creates a NEW ReplicaSet for v2 and gradually scales it up while scaling the OLD ReplicaSet down to zero. The old ReplicaSet is retained at 0 replicas so you can instantly roll back. This is how Kubernetes achieves zero-downtime releases.

2.2 The selector the link a Deployment must have

A Deployment finds the Pods it owns through label matching. The `spec.selector.matchLabels` MUST match the labels in the Pod template (`spec.template.metadata.labels`). If the selector is missing or does not match, the manifest is invalid / the controller cannot manage its Pods. This is one of the most common manifest bugs.

```
# FlixHub web frontend  note how selector matches template labels
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flixfhub-web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flixfhub-web          # <-- MUST equal the label below
  template:
    metadata:
      labels:
        app: flixfhub-web       # <-- the Pods get this label
    spec:
      containers:
        - name: web
          image: flixfhub/web:1.4
          ports:
            - containerPort: 8080
```

2.3 Controlling rolling updates: strategy block

The `strategy` block controls how a rollout proceeds. `maxUnavailable` caps how many Pods may be down at once; `maxSurge` caps how many extra Pods may be created above the desired count during the rollout. Tuning these gives true zero-downtime: keep at least the original count serving while new Pods come up.

```
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1    # at most 1 pod down during the update
      maxSurge: 1          # at most 1 extra pod created during the update
```

2.4 Services giving Pods a stable address

Pods are ephemeral and get new IPs when recreated, so you never talk to a Pod IP directly. A Service provides a stable virtual IP and DNS name and load-balances across the Pods that match its selector. Inside a cluster you can always reach a Service by its name (Kubernetes DNS resolves the Service name to its ClusterIP).

Service type	What it does / when to use it
ClusterIP	Default. Internal-only virtual IP reachable only from inside the cluster. Use for internal/backend services that should NOT be public.
NodePort	Exposes the Service on a static port on every node's IP. A basic way to reach a service from outside, often used in dev or behind your own load balancer.
LoadBalancer	Asks the cloud provider (via the cloud-controller-manager) to provision an external load balancer with a public IP. Use to expose a service to the public internet.

```
# Internal API → ClusterIP ; public web → LoadBalancer
apiVersion: v1
kind: Service
metadata:
  name: flixhub-web
spec:
  type: LoadBalancer    # ClusterIP for internal-only services
  selector:
    app: flixhub-web      # MUST match the Pods' labels
  ports:
    - port: 80            # port the Service listens on
      targetPort: 8080    # port the container listens on
```

Common bug: “Service has no endpoints”

If a Service shows no endpoints even though Pods are Running, the usual cause is that the Service selector does not match the Pod labels. The Service only routes to Pods whose labels match its selector. Fix the labels/selector so they agree, and the endpoints appear.

Also remember: port is the Service's port; targetPort is the container's port. Mixing them up breaks routing too.

Service type	What it does / when to use it
Check yourself Section 2 Explain the Pod → ReplicaSet → Deployment relationship and what happens to the old ReplicaSet on an image change. Given a manifest missing its selector, identify the field and write the corrected block. Write a Deployment that runs N replicas with a safe rolling update (maxUnavailable / maxSurge). Choose the right Service type for a public service vs an internal one, and define ClusterIP / NodePort / LoadBalancer. Diagnose why a Service returns no endpoints though Pods are Running.	

3. Persistent Storage: PV, PVC & StorageClass

3.1 Why Pods need external storage

A container's own filesystem is ephemeral when a Pod is deleted or rescheduled to another node, anything written inside the container is gone. Databases and any stateful workload therefore need storage that lives independently of the Pod, so data survives Pod restarts and node moves. Kubernetes provides this through PersistentVolumes and PersistentVolumeClaims.

3.2 The three objects and who owns them

Object	What it is	Owned by
PersistentVolume (PV)	An actual piece of storage in the cluster (a real disk/share), provisioned statically by an admin or dynamically.	Cluster admin / provisioner
PersistentVolumeClaim (PVC)	A request for storage of a given size and access mode. It binds to a matching PV.	Application developer
StorageClass	A template describing a tier/type of storage and the provisioner that creates it on demand.	Cluster admin

Dynamic provisioning the modern default

Instead of an admin pre-creating PVs, a PVC names a StorageClass. When the PVC is created, the StorageClass's provisioner automatically creates a matching PV (and the real cloud disk) on demand and binds them. The developer just asks for storage; the PV appears. This is what "dynamic provisioning" means.

3.3 Worked example: attaching a disk to a database Pod

A PVC requests storage; the Pod references the PVC as a volume and mounts it into the container at the path where the database writes its data. Study this pattern you will need to adapt it.

```
# 1) The claim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: flixhub-db-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 20Gi
---
# 2) Using it inside the Pod spec
volumes:
  - name: db-storage
    persistentVolumeClaim:
      claimName: flixhub-db-pvc
containers:
  - name: database
    volumeMounts:
      - name: db-storage
        mountPath: /var/lib/data    # where the DB writes
```

3.4 Access modes who can mount, and how

Access mode	Meaning	Typical backing storage
ReadWriteOnce (RWO)	Mounted read-write by a single node at a time. Fine for one database Pod.	Block disk (EBS, GCP PD, Azure Disk)
ReadOnlyMany (ROX)	Mounted read-only by many nodes.	Shared file storage
ReadWriteMany (RWX)	Mounted read-write by many nodes simultaneously needed when several replicas on different nodes must all write the same files.	Networked file storage: NFS, CephFS, EFS, Azure Files

Key distinction (high-value)

A single block disk can only do ReadWriteOnce one node at a time. If multiple replicas spread across nodes must share a read-write directory, RWO will not work; you need ReadWriteMany backed by networked/shared storage (NFS, CephFS, a cloud file service). Recognising when RWO is insufficient and RWX is required is a classic exam point.

3.5 Reclaim policies what happens to data when a PVC is deleted

Policy	Behaviour when the PVC is deleted
Retain	The PV and its data are KEPT; the admin cleans up manually later. Data survives choose this for important/financial data so accidental deletion does not destroy it.
Delete	The underlying storage asset is deleted along with the PVC. Data is permanently lost. Convenient for scratch/temporary data.

Policy	Behaviour when the PVC is deleted
Recycle	Deprecated. Old behaviour that scrubbed the volume for reuse.

Check yourself Section 3

Explain the PV / PVC / StorageClass relationship and who owns each; describe how dynamic provisioning works.

Write a PVC for a database (size + access mode + class) and the volumes/volumeMounts snippet that attaches it.

Explain why ReadWriteOnce fails when several replicas on different nodes must share a writable directory, and which mode + storage type is required instead.

Compare Retain vs Delete vs Recycle and justify which to pick for data you cannot afford to lose.

4. DevOps vs MLOps

4.1 Definitions

- **DevOps** – a culture and set of practices that unite software development and IT operations to deliver and run software quickly and reliably, using automation, CI/CD, and monitoring. The artifact being shipped is code.
- **MLOps** – an extension of DevOps to the machine-learning lifecycle. It keeps everything DevOps does, then adds the management of data and trained models on top, because an ML system is defined by its data and learned model, not just its code.

4.2 How MLOps differs from traditional DevOps

Dimension	DevOps (e.g. a web API)	MLOps (e.g. a recommendation model)
What is versioned	Source code	Code AND data AND the trained model artifact
What triggers a redeploy	A code change	A code change, OR new/changed data, OR a retrained model even with no code change
What “correctness” means	Passes tests; behaves as coded	Statistical quality (accuracy, precision) on live data, which can silently degrade over time
What you monitor	Uptime, latency, errors	All of that PLUS ongoing prediction quality

4.3 Why a model degrades without any code change: drift

A model is essentially a function learned from past data. The code can be frozen, yet the model can still get worse over time because the world changes. This is the central reason MLOps needs continuous retraining and monitoring.

- **Data drift** – the distribution of incoming input data shifts away from the data the model was trained on (e.g. user behaviour changes, new products appear). The inputs no longer look like the training set.
- **Model / concept drift** – the underlying relationship between inputs and the correct output changes, so the model's learned logic becomes stale even if the inputs look similar.

Why monitoring uptime is not enough

A model service can be perfectly ‘up’ returning HTTP 200s on time while quietly making poor predictions because of drift. So MLOps monitors prediction quality (accuracy/precision against real outcomes), not just availability. When quality drops, the model is retrained on fresh data and redeployed, even though no code changed.

4.4 What an MLOps pipeline adds beyond standard CI/CD

A normal code-only CI/CD pipeline (build → test → deploy) cannot capture the things that make an ML system reproducible and trustworthy. MLOps adds, among others:

- Data & experiment tracking versioning datasets and recording each training run's parameters and metrics, so results are reproducible.
- A model registry storing, versioning, and promoting trained model artifacts through stages (e.g. staging → production).
- Data/model validation gates checking incoming data quality and model performance before promotion.
- Continuous training & monitoring triggering retraining on drift or schedule, and watching live model quality.

Check yourself Section 4

Define DevOps and MLOps and give three concrete ways MLOps extends DevOps (versioning, redeploy triggers, correctness).

Explain why a model may need retraining/redeploying with no code change; define data drift and model drift.

Explain why monitoring quality matters, not just uptime.

Name things a code-only CI/CD pipeline cannot capture that an MLOps pipeline must add.

5. One-Page Quick Revision

Architecture

- Control plane = brain: apiserver (hub), etcd (state store), scheduler (placement), controller-manager (control loops), cloud-controller-manager (cloud integration).
- Worker node = muscle: kubelet (node agent + heartbeat), kube-proxy (networking), container runtime (runs containers).
- Losing etcd with no backup = losing the whole cluster's desired state.

Workloads

- Deployment manages ReplicaSet manages Pods. Image change → new ReplicaSet up, old scaled to 0 (kept for rollback).
- `spec.selector.matchLabels` must equal the Pod template labels. `strategy.rollingUpdate:` `maxUnavailable` / `maxSurge` control zero-downtime rollouts.

Services

- ClusterIP = internal only; NodePort = static port on every node; LoadBalancer = external cloud LB (public).
- `port` = Service port, `targetPort` = container port. No endpoints → selector/labels mismatch.

Storage

- PV = real storage; PVC = request that binds to a PV; StorageClass = template enabling dynamic provisioning.
- RWO = one node read-write; RWX = many nodes read-write (needs NFS/CephFS/cloud file). Reclaim: Retain keeps data, Delete destroys it.

DevOps vs MLOps

- DevOps ships code; MLOps also versions data + models, retrains on drift, and monitors prediction quality not just uptime.
- Data drift = inputs shift; model/concept drift = input→output relationship shifts. MLOps adds experiment tracking + model registry.

— End of Handout Good luck! —